```
} else {
  val prevState = state.get
  val newTimestamp = Instant.now
  val stateWithRecordsDropped = dropExpiredRecords(newTimestamp,
    prevState.currentTransactions)
  val output = new TransactionCount(CustomerID,
    stateWithRecordsDropped.currentTransactions.size,
    stateWithRecordsDropped.latestTimestamp, true)

  transactionCounts.append(output)
  state.update(stateWithRecordsDropped)
  state.setTimeoutTimestamp(
    stateWithRecordsDropped.latestTimestamp.toEpochMilli(),
    "30 seconds")
}
```

Figure 5.20 – If the state has timed out, the updateState function receives no records and only updates the state after dropping expired records

Note that the `transactionCounts` list buffer created at the beginning of the `updateState` function is returned as an iterator. In this case, the output will contain one record: the transaction count record for the specific customer.

Next, as we prepare to create a read stream, we define the input schema we need for our read stream; see *Figure 5.21*:

```
import org.apache.spark.sql.types.{StringType,
       StructField, StructType, IntegerType,
       FloatType, TimestampType}

// The schema for the incoming records
val schema = StructType(Array(
        StructField("Source", StringType, true),
        StructField("TransactionTimestamp", StringType, true),
        StructField("CustomerID", IntegerType, true),
        StructField("Amount", FloatType, true),
        StructField("Product", StringType, true),
        StructField("Label", IntegerType, true),
                ))
```

Figure 5.21 – Read stream input schema